

Web 开发技术 @ 2025

章许帆

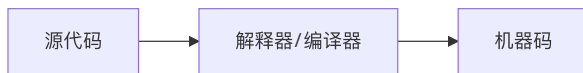
xufan.zhang@outlook.com

Web 前端开发技术



Web 前端开发技术

以工程化的方式将编写、维护、发布前端 HTML, CSS 和 JavaScript 组件



前端产物是什么？

前端产物

- ├─ *.html
- ├─ *.css
- └─ *.js

围绕如何以工程化的方法，遵循一系列最佳实践，展开前端开发过程，得到高质量的前端产物展开。

多页应用 (Multi-Plage Application, MPA)

通过跳转不同的 HTML 页面完成内容切换，在切换时，HTML、CSS 和 JavaScript 均需要重新加载

Apache/Nginx 中部署的网站目录结构：

```
├─ index.html      # 入口文件
├─ index.css       # 全局样式
├─ pages/          # 多页面目录
│   └─ home/       # 首页
│       ├── home.js    # 页面逻辑
│       ├── home.html  # 页面模板
│       └─ home.css    # 页面样式
│   └─ about/       # 关于页
│       ├── about.js   #
│       ├── about.html #
│       └─ about.css   #
│   └─ contact/      # 联系页
│       ├── contact.js #
│       ├── contact.html #
│       └─ contact.css #
└─ utils/           # 公共工具库
    └─ api.js        # 网络请求封装
```


单页应用 (Single-Page Application, SPA)

一种现代Web应用程序架构模式，它在单个HTML页面中动态重写当前页面而不是从服务器加载整个新页面

```
|— index.html      # 入口文件
|— index.css      # 全局样式
|— static/
|   |— home.js    # 页面逻辑
|   |— home.css   # 页面样式
|   |— about.js
|   |— about.css
|   |— contact.js
|   |— contact.css
|— utils/         # 公共工具库
|   |— api.js     # 网络请求封装
```

- 单页面结构: 整个应用只有一个HTML文件，内容区域动态更新
- 前后端分离: 前端负责UI渲染和路由，后端仅提供API接口
- 客户端渲染: 页面内容主要在浏览器中通过JavaScript生成
- 路由管理: 客户端路由处理URL变化而不刷新页面

如何得到 HTML + CSS + JavaScript

- Web 1.0 时代: 面向 UI 设计编程
- Web 2.0 -3.0 时代: 面向架构设计编程 + 工具打包 + AI 赋能

工具

- 项目管理、依赖管理工具 npm, 类似于 Maven, Gradle
- 项目构建工具 vite, 可以直接作为一个开发服务器
- 语法、格式检查工具 eslint, prettier
- 编程辅助工具 Trae, Github Copilot

目标: 不写 HTML, 少写CSS, JavaScript 搞定剩下的。

package.json

实现项目描述、依赖管理、脚本自动化等功能

■ 基础信息

```
{  
  "name": "project-name",      // 项目名称 (npm包名, 禁止大写和空格)  
  "version": "1.0.0",          // 版本号 (遵循语义化版本 semver)  
  "description": "A web app",  // 项目描述  
  "author": "Your Name",       // 作者  
  "license": "MIT",            // 开源协议  
  "private": true,             // 是否为私有项目 (禁止发布到npm)  
}
```

■ 脚本命令

```
{  
  "scripts": {  
    "start": "vite dev",        // 启动开发服务器  
    "build": "vite build",      // 生产环境构建  
    "test": "jest",            // 运行测试  
    "predeploy": "npm run build", // 钩子命令 (在deploy前自动执行)  
    "format": "prettier --write ." // 自定义工具命令  
  }  
}
```

package.json

实现项目描述、依赖管理、脚本自动化等功能

■ 依赖管理

```
{
  "dependencies": {           // 生产环境依赖（会打包到最终代码）
    "react": "^18.2.0",      // ^表示允许次要版本更新
    "lodash": "~4.17.21"     // ~表示允许补丁版本更新
  },
  "devDependencies": {       // 开发环境依赖（仅本地开发使用）
    "vite": "^4.0.0",        // 精确版本
    "eslint": "8.0.0"
  },
  "peerDependencies": {      // 宿主环境依赖（如插件库需要宿主提供）
    "react": ">=16.8.0"
  }
}
```

^18.2.0: 允许次要版本和补丁更新，即锁定大版本；~4.17.21: 仅允许补丁更新，即锁定小版本；8.0.0: 精确版本，不更新，即只能使用指定版本。

前端框架

	React	Vue	Angular
开发语言	JavaScript + JSX	JavaScript + 模板语法	TypeScript + 模板语法
数据绑定	单向（受控组件 + 状态提升）	双向（ <code>v-model</code> ） + 单向	双向（ <code>[(ngModel)]</code> ）
组件化	函数组件 + Hooks / Class	单文件组件（SFC）	装饰器（ <code>@Component</code> ）
状态管理	需外接（Redux/MobX）	内置（Vuex/Pinia）	内置（RxJS + Services）
路由	React Router	Vue Router	Angular Router
学习曲线	中等（需懂JS生态）	平缓（中文文档友好）	陡峭（TypeScript + 概念多）
性能优化	虚拟DOM + Fiber	虚拟DOM + 编译时优化	变更检测 + AOT编译

JSX VS. 模板语法

JSX (React) 和模板语法 (Vue, Angular 等) 是两种主流的 UI 构建方式

	JSX (React)	模板语法 (Vue, Angular)
本质	JavaScript 的扩展语法	基于 HTML 的增强语法
位置	与 JavaScript 逻辑写在一起	通常分离在单独的模板文件中
语法	类似 HTML 的 JavaScript	类似 HTML 但带有特殊指令

- 快速创建前端项目库:

```
npm create vite@latest my-react-app -- --template react
```

JSX VS. 模板语法

- React JSX 语法:

```
function Greeting() {  
  const message = "Greeting";  
  
  return (  
    <div>  
      <button>{message}</button>  
    </div>  
  )  
}
```

- Vue 模板语法:

```
<template>  
  <button>{{ message }}</button>  
</template>  
  
<script setup>  
  const message = "Greeting";  
</script>
```

React 组件

React App 的基本单元是 React Component

```
<button>Greeting</button>
```

■ React 组件

```
function Greeting() {  
  return (  
    <div>Hi </div>  
  )  
}
```

组件的使用和样式设置

```
1 function App() {  
2   return (  
3     <div className="container">  
4       <Greeting />  
5     </div>  
6   )  
7 }
```

React 组件

- React 组件也是一个 JavaScript 函数

```
1  const App = () => {  
2    return (  
3      <div>  
4        <Greeting />  
5      </div>  
6    )  
7  }
```

- React 组件绘制到页面

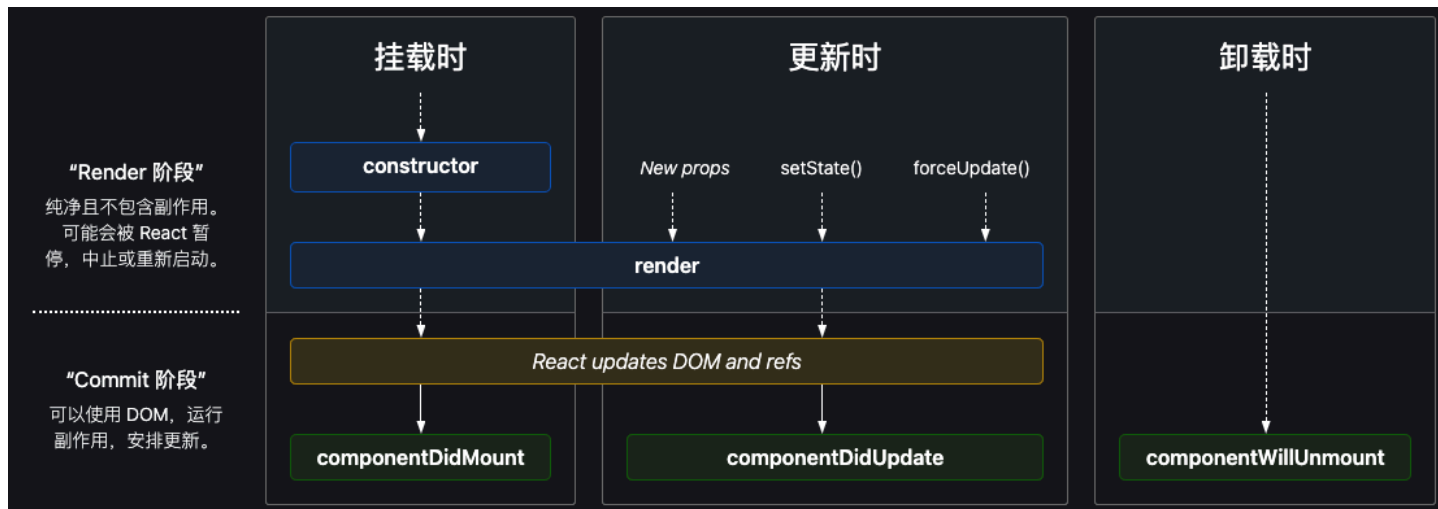
```
//main.jsx  
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <App />  
  </StrictMode>,  
)
```

JavaScript 操作 DOM

React 类组件

```
class Greeting extends React.Component {  
  render() {  
    return <div>Hello, {this.props.name}</div>  
  }  
}
```

与函数组件相比，类组件提供了状态和**生命周期**管理方法。



React 父子组件传参

使用标签的属性传递参数，使用 `props` 接收传递参数值

App.jsx

```
1  const App = () => {  
2    return (  
3      <div>  
4        <div>Welcome</div>  
5        <Greeting name="Zhangsan" />  
6      </div>  
7    )  
8  }
```

Greeting.jsx

```
1  function Greeting(props) {  
2    return (  
3      <div>Hi, {props.name} </div>  
4    )  
5  }
```

如何传递多个参数？

React 组件传参

- 使用在函数外部定义的变量

```
1  const name = 'Zhangsan';
2
3  function Greeting() {
4    return (
5      <div>Hi, {name}</div>
6    )
7  }
```

- 使用类组件状态值

```
1  class Greeting extends React.Component {
2    constructor() {
3      super();
4      this.state = {
5        name: 'Zhangsan'
6      }
7    }
8    render() {
9      return <div>Hello, {this.state.name}</div>
10    }
11  }
```

React 状态管理

在组件间存储、共享和同步动态数据的机制

什么是状态？ 组件记住的信息

类组件状态：

```
class Greeting extends React.Component {  
  constructor() {  
    super();  
    this.state = {  
      name: 'Zhangsan'  
    }  
  }  
  ...  
}
```

函数组件状态：

```
const App = () => {  
  const [ name, setName ] = useState('Zhangsan')  
}
```

React 全局状态管理

实现跨组件通信，针对频繁引起数据变化的场景，使用需格外谨慎

```
export const Context = createContext(); //定义全局上下文
```

```
function App() { // 父组件
  const [name, setName] = useState('zhangsan')

  return (
    <Context.Provider value={{name, setName}}>
      <div>
        <Greeting />
      </div>
    </Context.Provider>
  )
}
```

```
export function Greeting() { // 子组件
  const state = useContext(Context) // 获取上下文中的状态
  return (
    <div>Hi, {state.name}</div>
  )
}
```

React 页面重新渲染 (Re-rendering)

在自身状态、上级状态或外部数据变化时重新计算UI，但只有实际变化的部分才会更新到页面上。

重新渲染触发时机：

- 组件自身的状态 (State)
- 组件的 props 发生变化
- 父组件重新渲染
- 组件依赖的 Context 发生变化
- 强制重新渲染

状态 (State) 发生了变化，React 会重新渲染这个组件，以及这个组件所依赖的子组件。

中大型应用: Zustand, Redux

React 事件处理

HTML DOM 事件允许 Javascript 在 HTML 文档元素中注册不同事件处理程序

- 鼠标事件

onclick, ondblclick, onmouseover ...

- 键盘事件

onkeydown, onkeyup, onkeypress ...

- 表单事件

onchange, oninput, onsubmit ...

```
1  const btn = document.querySelector("button");
2
3  function greet(event) {
4    console.log("greet:", event);
5  }
6
7  btn.addEventListener("click", greet);
```

React 事件处理

React的事件处理系统提供了强大而一致的跨浏览器支持，理解其工作原理是构建响应式UI的基础

React 事件处理传递的是一个函数，最佳实践：

- 命名约定：处理器函数以"handle"开头
- 在构造函数或使用useCallback绑定this（类组件）
- 对于表单，使用受控组件模式
- 复杂交互考虑使用自定义Hook封装事件逻辑

```
1  function Greeting() {  
2  
3      function greet() {  
4          console.log("Hi");  
5      }  
6  
7      return (  
8          <button onClick={greet}>Greet</button>  
9      )  
10 }
```

React 路由

前端路由允许在不刷新整个页面的情况下，通过JavaScript动态地切换页面内容

工作原理: 监听 URL 变化，建立URL路径与UI组件的对应关系表，根据当前URL匹配对应的组件并渲染。

- Hash 模式

```
http://webdevelopment.com/#/frontend
```

Hash 模式下 Hash 的变化不会导致浏览器向服务器发起请求，通过监听浏览器的onhashchange() 事件，查找路由对应规则，刷新会加载到地址栏对应的页面

- History 模式

```
https://webdevelopment.com/frontend
```

History 模式利用 pushState() 和 replaceState() 方法改变 URL，刷新浏览器会重新发起请求，如果服务器没有匹配当前的 URL，会出现 404 Not Found。

React 路由

引入 react-router 或者 react-router-dom 实现对前端路由的支持

```
...  
import { BrowserRouter } from "react-router";  
  
const root = document.getElementById("root");  
  
ReactDOM.createRoot(root).render(  
  <BrowserRouter>  
    <App />  
  </BrowserRouter>  
);
```

路由信息配置:

```
<Routes>  
  <Route index element={<Home />} />  
  <Route path="about" element={<About />} />  
  <Route path=":profile" element={<Profile />} />  
</Routes>
```


Tailwind CSS

实用优先 (Utility-first) 的 CSS 框架，没有预定义的组件，而是提供了一系列原子化的 CSS 类

```
<!-- 传统CSS方式 -->
<div class="chat-notification"></div>
<style>
  .chat-notification {
    display: flex;
    padding: 1.5rem;
    border-radius: 0.5rem;
    background-color: #fff;
    box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.1);
  }
</style>

<!-- Tailwind方式 -->
<div class="flex p-6 rounded-lg bg-white shadow-lg"></div>
```

Tailwind CSS 类针对响应性进行优化，允许轻松定制，减少甚至消除对大型自定义 CSS 文件的需求，以提供更快的 UI 构建过程。

Tailwind CSS

- 安装相关依赖

```
npm install tailwindcss @tailwindcss/vite
```

- 添加配置

```
import tailwindcss from '@tailwindcss/vite'  
export default defineConfig({  
  plugins: [  
    tailwindcss(),  
  ],  
})
```

- 在 CSS 文件中导入

```
@import "tailwindcss";
```

- 在 HTML 文件中引入对应 CSS 文件

```
<html>  
  <link rel="stylesheet" href="index.css" />  
</html>
```

Tailwind CSS

display: inline, block, inline-block, flow-root, flex, inline-flex, grid, inline-grid ...

padding: p-<number>, p-px, ...

text-align: text-left, text-right, text-justify, ...

HTML Code Runner

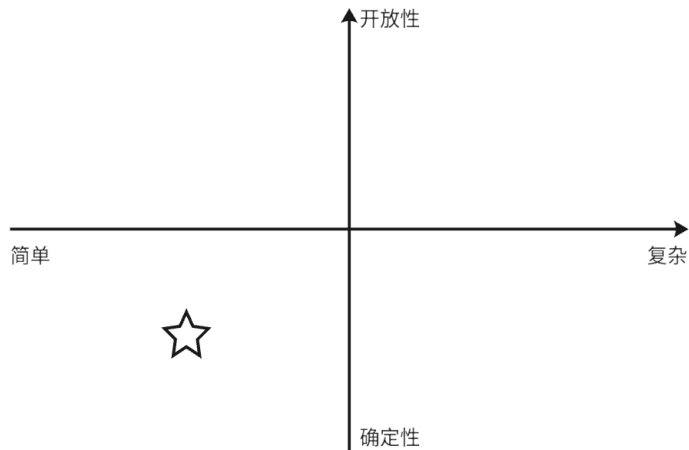
RESET

```
<div class="font-bold underline">Hello
```

输出

Hello Tailwind CSS

AI 能够帮助解决的问题



- 交互逻辑为主的问题
- 描述清晰的算法实现
- 逻辑相似的增删改查

大作业题目一

盲盒抽盒机

综合 新品 价格



CRYBABY搪胶毛绒挂件盲盒 豹豹猫猫系列

¥ 79 /油



THE MONSTERS 坐坐派对搪胶毛绒盲盒

¥ 99 /油



THE MONSTERS怪味便利店系列-耳机包

¥ 99 预售



THE MONSTERS 心动马卡龙搪胶脸盲盒

¥ 99 /油

首页 新品 商品 我的



LILIOS

岁·兔·旺系列 POP MART

暂无报价

灿若繁星
炫丽如火
愿2023人间充满烟火气
热闹祥和

岁·兔·旺

评论:4

用户beURox 06-27
好看

用户oHcJKo 03-08
欧气满满

睡不着的人 2023-01-07
新年快乐哦 互赞一个 欧气满满新的一年

Tinzi 作者 2023-01-08
欧气满满哦

说点什么吧 分享 4 6486

SKULLPANDA进退之门系列 预售 ¥69/油

我就是隐藏款 正在抽盒 共2人

道具场 切换



No.1022533288VQKK2AUXGC

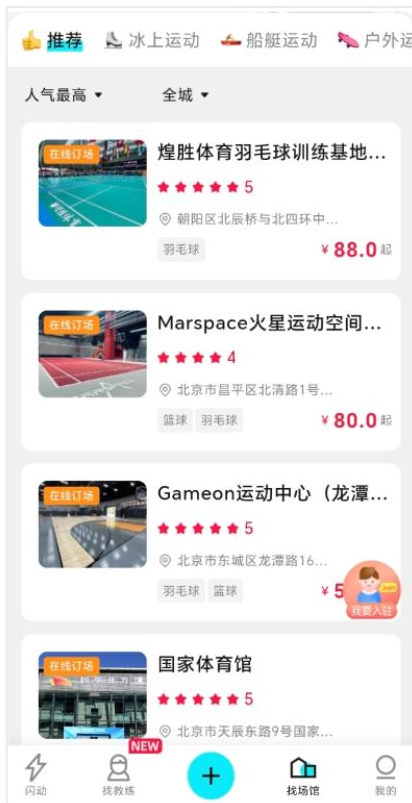
预售提醒: 预计 2025年08月04日00点 起开始发货

排队选盒

点击查看商品详情

大作业题目二

■ 体育活动室



作业要求（一）

- 可以使用 AI 助手，但你要对其输出负责
- 给你的 Web 应用起个独一无二的名字吧
- 针对你的选题，调研现有类似应用的功能
- 使用前后端分离的方式，前端采用 SPA 形式
- 前端建议使用 Vite + React + TailwindCSS
- 页面组件化，实现独立、可重用的小部件
- 基于 Github 进行代码管理和持续集成



谢谢